

Loan Approval Prediction

Import Libraries

```
In [1]: import numpy as np
import pandas as pd
```

Load The File

```
In [2]: df=pd.read_excel('Loan_Approval_Prediction_Dataset.xlsx')
print(df)
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP100001	Female	Yes	1	Not Graduate	No	
1	LP100002	Male	No	0	Graduate	No	
2	LP100003	Male	No	1	Graduate	No	
3	LP100004	Male	Yes	0	Not Graduate	Yes	
4	LP100005	Male	Yes	2	Not Graduate	Yes	
..	...	...	...	...	...	...	
995	LP100996	Female	Yes	2	Graduate	No	
996	LP100997	Female	Yes	0	Not Graduate	Yes	
997	LP100998	Male	No	2	Graduate	No	
998	LP100999	Male	No	1	Graduate	No	
999	LP101000	Female	No	0	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	12141	12484	659	360	
1	23323	3170	530	300	
2	7401	1362	409	300	
3	4170	3654	88	240	
4	22332	12376	109	240	
..	...	...	...	...	
995	9950	7915	562	300	
996	14085	3195	98	360	
997	12983	8516	183	300	
998	5907	2360	666	300	
999	20494	6205	60	240	

	Credit_History	Property_Area	Loan_Status
0	1	Urban	Y
1	0	Rural	N
2	0	Urban	N
3	0	Rural	N
4	0	Semiurban	N
..	...	...	...
995	1	Rural	Y
996	1	Semiurban	Y
997	1	Rural	Y
998	0	Urban	N
999	1	Urban	Y

[1000 rows x 13 columns]

Display the First 10 Rows

```
In [3]: df.head(10)
```

Out[3]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	Loan/
0	LP100001	Female	Yes	1	Not Graduate	No	12141		12484
1	LP100002	Male	No	0	Graduate	No	23323		3170
2	LP100003	Male	No	1	Graduate	No	7401		1362
3	LP100004	Male	Yes	0	Not Graduate	Yes	4170		3654
4	LP100005	Male	Yes	2	Not Graduate	Yes	22332		12376
5	LP100006	Female	No	3+	Graduate	Yes	8908		8986
6	LP100007	Male	Yes	3+	Graduate	No	17034		3548
7	LP100008	Male	Yes	1	Not Graduate	No	16539		9168
8	LP100009	Male	No	1	Not Graduate	No	15993		7225
9	LP100010	Female	Yes	1	Graduate	Yes	15617		7845

Display the last 10 Rows

In [4]:

df.tail(10)

Out[4]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	Loa
990	LP100991	Female	No	3+	Not Graduate	No	7176		3041
991	LP100992	Female	No	0	Graduate	No	1780		963
992	LP100993	Male	No	3+	Graduate	Yes	10769		168
993	LP100994	Male	Yes	2	Not Graduate	Yes	5251		13003
994	LP100995	Female	No	3+	Graduate	No	10016		1119
995	LP100996	Female	Yes	2	Graduate	No	9950		7915
996	LP100997	Female	Yes	0	Not Graduate	Yes	14085		3195
997	LP100998	Male	No	2	Graduate	No	12983		8516
998	LP100999	Male	No	1	Graduate	No	5907		2360
999	LP101000	Female	No	0	Graduate	Yes	20494		6205

Shape

In [5]:

print(df.shape)

(1000, 13)

Size

In [6]:

print(df.size)

13000

Information Of The dataset

In [7]:

df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID                1000 non-null   object
1   Gender                 1000 non-null   object
2   Married                1000 non-null   object
3   Dependents             1000 non-null   object
4   Education              1000 non-null   object
5   Self_Employed          1000 non-null   object
6   ApplicantIncome        1000 non-null   int64
7   CoapplicantIncome      1000 non-null   int64
8   LoanAmount             1000 non-null   int64
9   Loan_Amount_Term       1000 non-null   int64
10  Credit_History          1000 non-null   int64
11  Property_Area           1000 non-null   object
12  Loan_Status            1000 non-null   object
dtypes: int64(5), object(8)
memory usage: 101.7+ KB

```

Statistical Summary

```
In [8]: df.describe()
```

```
Out[8]:
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	13332.918000	7538.225000	375.182000	264.660000	0.492000
std	6905.149877	4290.910832	188.770525	67.117726	0.500186
min	1504.000000	35.000000	50.000000	180.000000	0.000000
25%	7396.250000	3837.750000	214.750000	180.000000	0.000000
50%	13334.000000	7405.000000	372.000000	240.000000	0.000000
75%	19367.250000	11195.250000	538.500000	300.000000	1.000000
max	24971.000000	14971.000000	700.000000	360.000000	1.000000

DATA CLEANING

Missing Values

```
In [9]: print(df.isnull())
# NOTE- IF DATA IS MISSING OUPUT WILL BE TRUE
# IF DATA IS NOT MISSING OUPUT WILL BE FALSE
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	False	False	False	False	False	False	
1	False	False	False	False	False	False	
2	False	False	False	False	False	False	
3	False	False	False	False	False	False	
4	False	False	False	False	False	False	
..	...	...	...	...	...	...	
995	False	False	False	False	False	False	
996	False	False	False	False	False	False	
997	False	False	False	False	False	False	
998	False	False	False	False	False	False	
999	False	False	False	False	False	False	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	False	False	False	False	
1	False	False	False	False	
2	False	False	False	False	
3	False	False	False	False	
4	False	False	False	False	
..	...	...	...	...	
995	False	False	False	False	
996	False	False	False	False	
997	False	False	False	False	
998	False	False	False	False	
999	False	False	False	False	

	Credit_History	Property_Area	Loan_Status
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False
..	...	...	...
995	False	False	False
996	False	False	False
997	False	False	False
998	False	False	False
999	False	False	False

[1000 rows x 13 columns]

Sum of the Missing Values

```
In [10]: print(df.isnull().sum())
```

```
Loan_ID          0
Gender           0
Married          0
Dependents       0
Education        0
Self_Employed    0
ApplicantIncome  0
CoapplicantIncome 0
LoanAmount       0
Loan_Amount_Term 0
Credit_History   0
Property_Area    0
Loan_Status      0
dtype: int64
```

Label Encoding

```
In [11]: from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df['Gender']=le.fit_transform(df['Gender'])
print(df)
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP100001	0	Yes	1	Not Graduate	No	
1	LP100002	1	No	0	Graduate	No	
2	LP100003	1	No	1	Graduate	No	
3	LP100004	1	Yes	0	Not Graduate	Yes	
4	LP100005	1	Yes	2	Not Graduate	Yes	
..	...	...	...	...	...	...	
995	LP100996	0	Yes	2	Graduate	No	
996	LP100997	0	Yes	0	Not Graduate	Yes	
997	LP100998	1	No	2	Graduate	No	
998	LP100999	1	No	1	Graduate	No	
999	LP101000	0	No	0	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	12141	12484	659	360	
1	23323	3170	530	300	
2	7401	1362	409	300	
3	4170	3654	88	240	
4	22332	12376	109	240	
..	...	...	...	...	
995	9950	7915	562	300	
996	14085	3195	98	360	
997	12983	8516	183	300	
998	5907	2360	666	300	
999	20494	6205	60	240	

	Credit_History	Property_Area	Loan_Status
0	1	Urban	Y
1	0	Rural	N
2	0	Urban	N
3	0	Rural	N
4	0	Semiurban	N
..	...	...	...
995	1	Rural	Y
996	1	Semiurban	Y
997	1	Rural	Y
998	0	Urban	N
999	1	Urban	Y

[1000 rows x 13 columns]

```
In [12]: mapping = dict(zip(le.classes_, le.transform(le.classes_)))
          print(mapping)

{'Female': np.int64(0), 'Male': np.int64(1)}
```

```
In [13]: df['Married']=le.fit_transform(df['Married'])
          print(df)
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP100001	0	1	1	Not Graduate	No	
1	LP100002	1	0	0	Graduate	No	
2	LP100003	1	0	1	Graduate	No	
3	LP100004	1	1	0	Not Graduate	Yes	
4	LP100005	1	1	2	Not Graduate	Yes	
..	...	...	...	...	...	...	
995	LP100996	0	1	2	Graduate	No	
996	LP100997	0	1	0	Not Graduate	Yes	
997	LP100998	1	0	2	Graduate	No	
998	LP100999	1	0	1	Graduate	No	
999	LP101000	0	0	0	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	12141	12484	659	360	
1	23323	3170	530	300	
2	7401	1362	409	300	
3	4170	3654	88	240	
4	22332	12376	109	240	
..	...	...	...	...	
995	9950	7915	562	300	
996	14085	3195	98	360	
997	12983	8516	183	300	
998	5907	2360	666	300	
999	20494	6205	60	240	

	Credit_History	Property_Area	Loan_Status
0	1	Urban	Y
1	0	Rural	N
2	0	Urban	N
3	0	Rural	N
4	0	Semiurban	N
..	...	...	...
995	1	Rural	Y
996	1	Semiurban	Y
997	1	Rural	Y
998	0	Urban	N
999	1	Urban	Y

[1000 rows x 13 columns]

```
In [14]: mapping = dict(zip(le.classes_, le.transform(le.classes_)))
          print(mapping)

{'No': np.int64(0), 'Yes': np.int64(1)}
```

```
In [15]: df['Dependents']=le.fit_transform(df['Dependents'])
          print(df)
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP100001	0	1	1	Not Graduate	No	
1	LP100002	1	0	0	Graduate	No	
2	LP100003	1	0	1	Graduate	No	
3	LP100004	1	1	0	Not Graduate	Yes	
4	LP100005	1	1	2	Not Graduate	Yes	
..	...	...	...	...	...	...	
995	LP100996	0	1	2	Graduate	No	
996	LP100997	0	1	0	Not Graduate	Yes	
997	LP100998	1	0	2	Graduate	No	
998	LP100999	1	0	1	Graduate	No	
999	LP101000	0	0	0	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	12141	12484	659	360	
1	23323	3170	530	300	
2	7401	1362	409	300	
3	4170	3654	88	240	
4	22332	12376	109	240	
..	...	...	...	...	
995	9950	7915	562	300	
996	14085	3195	98	360	
997	12983	8516	183	300	
998	5907	2360	666	300	
999	20494	6205	60	240	

	Credit_History	Property_Area	Loan_Status
0	1	Urban	Y
1	0	Rural	N
2	0	Urban	N
3	0	Rural	N
4	0	Semiurban	N
..	...	...	...
995	1	Rural	Y
996	1	Semiurban	Y
997	1	Rural	Y
998	0	Urban	N
999	1	Urban	Y

[1000 rows x 13 columns]

```
In [16]: mapping = dict(zip(le.classes_, le.transform(le.classes_)))
          print(mapping)

{'0': np.int64(0), '1': np.int64(1), '2': np.int64(2), '3+': np.int64(3)}
```

```
In [17]: df['Education']=le.fit_transform(df['Education'])
          print(df)
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP100001	0	1	1	1	No	
1	LP100002	1	0	0	0	No	
2	LP100003	1	0	1	0	No	
3	LP100004	1	1	0	1	Yes	
4	LP100005	1	1	2	1	Yes	
..	...	...	...	...	...	...	
995	LP100996	0	1	2	0	No	
996	LP100997	0	1	0	1	Yes	
997	LP100998	1	0	2	0	No	
998	LP100999	1	0	1	0	No	
999	LP101000	0	0	0	0	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	12141	12484	659	360	
1	23323	3170	530	300	
2	7401	1362	409	300	
3	4170	3654	88	240	
4	22332	12376	109	240	
..	...	...	...	...	
995	9950	7915	562	300	
996	14085	3195	98	360	
997	12983	8516	183	300	
998	5907	2360	666	300	
999	20494	6205	60	240	

	Credit_History	Property_Area	Loan_Status
0	1	Urban	Y
1	0	Rural	N
2	0	Urban	N
3	0	Rural	N
4	0	Semiurban	N
..	...	...	...
995	1	Rural	Y
996	1	Semiurban	Y
997	1	Rural	Y
998	0	Urban	N
999	1	Urban	Y

[1000 rows x 13 columns]

```
In [18]: mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print(mapping)
```

```
{'Graduate': np.int64(0), 'Not Graduate': np.int64(1)}
```

```
In [19]: df['Self_Employed']=le.fit_transform(df['Self_Employed'])
print(df)
```


	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP100001	0	1	1	1	0	
1	LP100002	1	0	0	0	0	
2	LP100003	1	0	1	0	0	
3	LP100004	1	1	0	1	1	
4	LP100005	1	1	2	1	1	
..	...	...	...	...	...	...	
995	LP100996	0	1	2	0	0	
996	LP100997	0	1	0	1	1	
997	LP100998	1	0	2	0	0	
998	LP100999	1	0	1	0	0	
999	LP101000	0	0	0	0	1	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	12141	12484	659	360	
1	23323	3170	530	300	
2	7401	1362	409	300	
3	4170	3654	88	240	
4	22332	12376	109	240	
..	...	...	...	...	
995	9950	7915	562	300	
996	14085	3195	98	360	
997	12983	8516	183	300	
998	5907	2360	666	300	
999	20494	6205	60	240	

	Credit_History	Property_Area	Loan_Status
0	1	Urban	Y
1	0	Rural	N
2	0	Urban	N
3	0	Rural	N
4	0	Semiurban	N
..	...	...	...
995	1	Rural	Y
996	1	Semiurban	Y
997	1	Rural	Y
998	0	Urban	N
999	1	Urban	Y

[1000 rows x 13 columns]

```
In [20]: mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print(mapping)

{'No': np.int64(0), 'Yes': np.int64(1)}
```

```
In [21]: df['Property_Area']=le.fit_transform(df['Property_Area'])
print(df)
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP100001	0	1	1	1	0	
1	LP100002	1	0	0	0	0	
2	LP100003	1	0	1	0	0	
3	LP100004	1	1	0	1	1	
4	LP100005	1	1	2	1	1	
..	...	...	...	...	...	...	
995	LP100996	0	1	2	0	0	
996	LP100997	0	1	0	1	1	
997	LP100998	1	0	2	0	0	
998	LP100999	1	0	1	0	0	
999	LP101000	0	0	0	0	1	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	12141	12484	659	360	
1	23323	3170	530	300	
2	7401	1362	409	300	
3	4170	3654	88	240	
4	22332	12376	109	240	
..	...	...	...	...	
995	9950	7915	562	300	
996	14085	3195	98	360	
997	12983	8516	183	300	
998	5907	2360	666	300	
999	20494	6205	60	240	

	Credit_History	Property_Area	Loan_Status
0	1	2	Y
1	0	0	N
2	0	2	N
3	0	0	N
4	0	1	N
..	...	...	...
995	1	0	Y
996	1	1	Y
997	1	0	Y
998	0	2	N
999	1	2	Y

[1000 rows x 13 columns]

```
In [22]: mapping = dict(zip(le.classes_, le.transform(le.classes_)))
          print(mapping)

{'Rural': np.int64(0), 'Semiurban': np.int64(1), 'Urban': np.int64(2)}

In [23]: df['Loan_Status']=le.fit_transform(df['Loan_Status'])
          print(df)
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP100001	0	1	1	1	0	
1	LP100002	1	0	0	0	0	
2	LP100003	1	0	1	0	0	
3	LP100004	1	1	0	1	1	
4	LP100005	1	1	2	1	1	
...	...	...	...	...	...	...	
995	LP100996	0	1	2	0	0	
996	LP100997	0	1	0	1	1	
997	LP100998	1	0	2	0	0	
998	LP100999	1	0	1	0	0	
999	LP101000	0	0	0	0	1	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	12141	12484	659	360	
1	23323	3170	530	300	
2	7401	1362	409	300	
3	4170	3654	88	240	
4	22332	12376	109	240	
...	...	...	...	...	
995	9950	7915	562	300	
996	14085	3195	98	360	
997	12983	8516	183	300	
998	5907	2360	666	300	
999	20494	6205	60	240	

	Credit_History	Property_Area	Loan_Status
0	1	2	1
1	0	0	0
2	0	2	0
3	0	0	0
4	0	1	0
...	...	...	...
995	1	0	1
996	1	1	1
997	1	0	1
998	0	2	0
999	1	2	1

[1000 rows x 13 columns]

```
In [24]: mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print(mapping)

{'N': np.int64(0), 'Y': np.int64(1)}
```

```
In [25]: df.head(10)
```

```
Out[25]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	Loan/
0	LP100001	0	1	1	1	0	12141	12484	
1	LP100002	1	0	0	0	0	23323	3170	
2	LP100003	1	0	1	0	0	7401	1362	
3	LP100004	1	1	0	1	1	4170	3654	
4	LP100005	1	1	2	1	1	22332	12376	
5	LP100006	0	0	3	0	1	8908	8986	
6	LP100007	1	1	3	0	0	17034	3548	
7	LP100008	1	1	1	1	0	16539	9168	
8	LP100009	1	0	1	1	0	15993	7225	
9	LP100010	0	1	1	0	1	15617	7845	

```
In [26]: X=df.drop(['Loan_ID', 'Loan_Status'],axis=1)
y=df['Loan_Status']
```

Train - Test Split data

```
In [27]: from sklearn.model_selection import train_test_split
X=df.drop(['Loan_ID', 'Loan_Status'],axis=1)
y=df['Loan_Status']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

Feature Scalling

```
In [28]: from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
```

Logistic Regression

```
In [29]: from sklearn.linear_model import LogisticRegression
model=LogisticRegression(random_state=42)
model.fit(X_train,y_train)
```

```
Out[29]: ▼ LogisticRegression ⓘ ?
          ► Parameters
```

```
In [30]: y_pred=model.predict(X_test)
print(y_pred)

[1 0 0 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0 1 1 1 1 1 0 0 1 1 1 0 0 1 0 0
 0 0 0 0 1 1 1 1 1 0 0 0 1 1 0 0 0 0 1 0 1 1 1 1 0 0 0 0 1 0 0 0 1 1 0 1 1
 1 1 0 1 1 1 0 0 0 1 1 1 0 1 0 0 0 0 1 0 0 0 1 1 1 1 0 0 1 0 0 1 0 0 0 0 1
 0 0 1 0 0 0 0 1 0 0 1 1 1 1 0 1 0 0 0 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 0 1 0 0 1 1 0 0 1 1 1 1 0 0 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1
 0 0 0 1 0 0 1 0 1 0 1 1 0 0 1]
```

```
In [31]: from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 0.92

```
In [32]: from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[ 75   5]
 [ 11 109]]
```

```
In [33]: from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.87	0.94	0.90	80
1	0.96	0.91	0.93	120
accuracy			0.92	200
macro avg	0.91	0.92	0.92	200
weighted avg	0.92	0.92	0.92	200

```
In [34]: import pandas as pd
result=pd.DataFrame({
    'actual':y_test.values,
    'predicted':y_pred
})
print(result)
result.head(10)
```

	actual	predicted
0	1	1
1	0	0
2	0	0
3	0	0
4	1	1
..	...	...
195	1	1
196	1	1
197	0	0
198	0	0
199	1	1

[200 rows x 2 columns]

```
Out[34]:
```

	actual	predicted
0	1	1
1	0	0
2	0	0
3	0	0
4	1	1
5	0	0
6	1	1
7	1	1
8	1	1
9	0	1

```
In [35]: Gender=int(input('Gender:'))
Married=int(input('Married Status:'))
Dependents=int(input('Dependents:'))
Education=int(input('Education Status:'))
Self_Employed=int(input('Self_Employed:'))
ApplicantIncome=int(input('Enter the Applicant Income:'))
CoapplicantIncome=int(input('Enter the Co Applicant Income:'))
LoanAmount=int(input('Enter the Loan Amounr:'))
Loan_Amount_Term=int(input('Enter the Loan Amount Term:'))
Credit_History=int(input('Enter the Credit History:'))
Property_Area=int(input('Enter the Property Area:'))
```

```
In [36]: import pandas as pd
user_data = pd.DataFrame({
    "Gender": [Gender],
    "Married": [Married],
    "Dependents": [Dependents],
    "Education": [Education],
    "Self_Employed": [Self_Employed],
    "ApplicantIncome": [ApplicantIncome],
    "CoapplicantIncome": [CoapplicantIncome],
    "LoanAmount": [LoanAmount],
    "Loan_Amount_Term": [Loan_Amount_Term],
    "Credit_History": [Credit_History],
    "Property_Area": [Property_Area]
})
print(user_data)
```

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome \
0	1	0	2	0	1	25000
	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	\	
0	6000	250	360	1		
	Property_Area					
0	2					

```
In [37]: user_data=scaler.transform(user_data)
print(user_data)
```

```
[[ 1.0100505 -0.98757716  0.48541884 -0.98757716  1.03563452  1.69111817
 -0.3571569 -0.66398948  1.4505428  1.03823026  1.18990995]]
```

```
In [38]: import joblib

joblib.dump(model, "loan_approval_model.pkl")
```

```
Out[38]: ['loan_approval_model.pkl']
```

```
In [39]: joblib.dump(scaler, "loan_approval_scaler.pkl")
```

```
Out[39]: ['loan_approval_scaler.pkl']
```

```
In [40]: import joblib

model = joblib.load("loan_approval_model.pkl")
scaler = joblib.load("loan_approval_scaler.pkl")
```

```
In [41]: prediction = model.predict(user_data)

if prediction[0] == 1:
```

```
    print("Loan Approved")  
else:  
    print("Loan Rejected")
```

Loan Approved

In []: